SQL Query Reference

All queries run via DuckDB in-process against mf.duckdb · Scripts: 02_funnel.py 03_cohorts.py 04_rfm.py

A — DATA LOADING (02_funnel.py)

Q1
02_funnel.py

Load CSVs into DuckDB. read_csv_auto() sniffs column types automatically; empty date strings become NULL. Safe to re-run — tables are dropped first.

```
DROP TABLE IF EXISTS users;  DROP TABLE IF EXISTS events;
CREATE TABLE users  AS SELECT * FROM read_csv_auto('data/users.csv',  header=true);
CREATE TABLE events AS SELECT * FROM read_csv_auto('data/events.csv', header=true);
```

B — ACTIVATION FUNNEL (02_funnel.py)

Q2
02_funnel.py

Six-stage funnel via stacked CTEs — one CTE per stage, assembled with UNION ALL. Stages: signup, KYC, first investment, SIP setup, second investment within 30 days, active at Day 90.

```
WITH s1 AS (SELECT COUNT(DISTINCT user_id) AS n FROM users),
s2 AS (SELECT COUNT(DISTINCT user_id) AS n FROM users WHERE kyc_completed_date IS NOT NULL),
s3 AS (SELECT COUNT(DISTINCT user_id) AS n FROM events WHERE event_type='first_investment'),
s4 AS (SELECT COUNT(DISTINCT user_id) AS n FROM events WHERE event_type='sip_setup'),
first_inv AS (SELECT user_id, MIN(event_date) AS first_date FROM events
              WHERE event_type='first_investment' GROUP BY user_id),
s5 AS (SELECT COUNT(DISTINCT e.user_id) AS n FROM events e
       JOIN first_inv fi ON e.user_id=fi.user_id
       WHERE e.event_type IN ('lumpsum','sip_installment','sip_setup')
         AND e.event_date > fi.first_date AND e.event_date <= fi.first_date + INTERVAL 30 DAY),
s6 AS (SELECT COUNT(DISTINCT e.user_id) AS n FROM events e JOIN users u ON e.user_id=u.user_id
       WHERE CAST(e.event_date AS DATE) BETWEEN CAST(u.signup_date AS DATE)+INTERVAL 75 DAY
                                           AND CAST(u.signup_date AS DATE)+INTERVAL 105 DAY)
SELECT 1,'Signed Up',(SELECT n FROM s1) UNION ALL SELECT 2,'KYC',(SELECT n FROM s2)
UNION ALL SELECT 3,'First Investment',(SELECT n FROM s3)
UNION ALL SELECT 4,'SIP Setup',(SELECT n FROM s4)
UNION ALL SELECT 5,'2nd Invest <=30d',(SELECT n FROM s5)
UNION ALL SELECT 6,'Active Day 90',(SELECT n FROM s6) ORDER BY 1;
```

Q3
02_funnel.py

Signup-to-first-investment conversion rate by source_channel. LEFT JOIN ensures channels with zero investors still appear.

```
WITH channel_signups AS (SELECT source_channel, COUNT(DISTINCT user_id) AS signed_up FROM users GROUP BY 1),
channel_investors AS (SELECT u.source_channel, COUNT(DISTINCT e.user_id) AS invested
  FROM events e JOIN users u ON e.user_id=u.user_id WHERE e.event_type='first_investment' GROUP BY 1)
SELECT cs.source_channel, cs.signed_up, COALESCE(ci.invested,0) AS made_first_investment,
  ROUND(COALESCE(ci.invested,0)*100.0/cs.signed_up,1) AS conversion_pct
FROM channel_signups cs LEFT JOIN channel_investors ci ON cs.source_channel=ci.source_channel
ORDER BY conversion_pct DESC;
```

C — RETENTION COHORTS (03_cohorts.py)

Q4
03_cohorts.py

Weekly cohort retention matrix. CROSS JOINs users x weeks 0–12 to build all (user, week) pairs, then LEFT JOINs events to flag activity in each 7-day window. Aggregates to cohort level for a 19-cohort x 13-week result.

```
WITH user_cohorts AS (SELECT user_id, signup_date, STRFTIME(signup_date,'%Y-%m') AS signup_month
  FROM users WHERE signup_date IS NOT NULL),
weeks_spine AS (SELECT UNNEST(RANGE(0,13)) AS week_num),
user_weeks AS (SELECT uc.user_id, uc.signup_date, uc.signup_month, ws.week_num
  FROM user_cohorts uc CROSS JOIN weeks_spine ws),
activity_flags AS (
  SELECT uw.signup_month, uw.week_num, uw.user_id,
    MAX(CASE WHEN e.event_date >= uw.signup_date + INTERVAL (uw.week_num*7) DAY
             AND e.event_date <  uw.signup_date + INTERVAL ((uw.week_num+1)*7) DAY
             AND e.event_type IN ('first_investment','lumpsum','sip_setup','sip_installment')
            THEN 1 ELSE 0 END) AS was_active
  FROM user_weeks uw LEFT JOIN events e ON e.user_id=uw.user_id
  GROUP BY uw.signup_month, uw.week_num, uw.user_id),
cohort_weekly AS (SELECT signup_month, week_num AS week_number,
  COUNT(DISTINCT user_id) AS cohort_size, SUM(was_active) AS active_users,
  ROUND(SUM(was_active)*100.0/NULLIF(COUNT(DISTINCT user_id),0),1) AS retention_pct
  FROM activity_flags GROUP BY signup_month, week_num)
SELECT signup_month, week_number, cohort_size, CAST(active_users AS INTEGER), retention_pct
FROM cohort_weekly ORDER BY signup_month, week_number;
```

D — RFM SEGMENTATION (04_rfm.py)

Q5
04_rfm.py

Compute raw R, F, M per investor. Recency = days since last investment (all-time, from dataset max date). Frequency and Monetary scoped to the last 180 days — using MAX(event_date) not today ensures reproducibility.

```
WITH ref_date AS (SELECT MAX(event_date) AS today FROM events),
invest_events AS (SELECT user_id, event_date, amount FROM events
  WHERE event_type IN ('first_investment','lumpsum','sip_setup','sip_installment')),
recency_cte AS (SELECT ie.user_id,
  DATE_DIFF('day', MAX(ie.event_date), rd.today) AS recency_days
  FROM invest_events ie CROSS JOIN ref_date rd GROUP BY ie.user_id, rd.today),
freq_monetary_cte AS (SELECT ie.user_id, COUNT(*) AS frequency, SUM(ie.amount) AS monetary
  FROM invest_events ie CROSS JOIN ref_date rd
  WHERE ie.event_date >= rd.today - INTERVAL 180 DAY GROUP BY ie.user_id)
SELECT r.user_id, r.recency_days,
  COALESCE(fm.frequency,0) AS frequency, COALESCE(fm.monetary,0.0) AS monetary
FROM recency_cte r LEFT JOIN freq_monetary_cte fm ON r.user_id=fm.user_id ORDER BY r.user_id;
```

E — PYTHON POST-PROCESSING (04_rfm.py)

P1
04_rfm.py

Quintile scoring (1–5) via pd.qcut(). R is reversed: fewer days = score 5. rank(method='first') breaks ties before qcut. Segment assigned by priority-ordered if/elif chain — first matching rule wins.

```
rfm['R'] = pd.qcut(rfm['recency_days'], q=5, labels=[5,4,3,2,1], duplicates='drop').astype(int)
rfm['F'] = pd.qcut(rfm['frequency'].rank(method='first'), q=5, labels=[1,2,3,4,5], duplicates='drop').astype(int)
rfm['M'] = pd.qcut(rfm['monetary'].rank(method='first'),  q=5, labels=[1,2,3,4,5], duplicates='drop').astype(int)
def assign_segment(row):
    R,F = row['R'],row['F']
    if   R>=4 and F>=4:          return 'Champions'
    elif R>=3 and F>=3:          return 'Loyal'
    elif R>=3 and F<=3:          return 'Potential Loyalist'
    elif R>=4 and F<=2:          return 'New Investor'
    elif 2<=R<=3 and F>=3:       return 'At Risk'
    elif 1<=R<=2 and 2<=F<=3:    return 'Hibernating'
    else:                         return 'Lost'
```

F — QUERY SUMMARY

Query	Script	Output rows	Purpose
Q1 — Load CSVs	02_funnel.py	50K + 82K rows	Ingest CSVs into DuckDB tables
Q2 — Activation Funnel	02_funnel.py	6 rows (one per stage)	User count surviving each funnel stage
Q3 — Channel Breakdown	02_funnel.py	5 rows (one per channel)	Conversion rate by acquisition source
Q4 — Cohort Matrix	03_cohorts.py	247 rows (19mo x 13wk)	Weekly retention % per signup cohort
Q5 — RFM Raw Values	04_rfm.py	22,214 rows (one per investor)	R / F / M values per investor
P1 — RFM Scoring	04_rfm.py	22,214 rows + segment col	Quintile scores + named segment

All SQL executed via DuckDB 0.9+ in-process (no server required). Scoring (P1) uses pandas 2.x pd.qcut() — quintile boundaries shift if the investor population changes. Re-running against updated data may reassign segment labels.